

FinAlze Backend

Full Capability Description

Multi-tenant, async, partner-facing financial-document extraction service

The backend turns the in-process OCR pipeline into a multi-tenant extraction service. A slim FastAPI API publishes jobs by name to a Celery / RabbitMQ broker; a separate worker (with the heavy torch / docling pipeline) executes them and writes results back to a shared operational store. The API never imports the pipeline, keeping it lightweight.

1. Document Extraction (async job pipeline)

- **POST /api/v1/documents** — multipart upload → stored → extraction job enqueued; returns 202 with `job_id` / `document_id`. Streamed read with a size cap (default 25 MB) and an extension allowlist (pdf, docx, jpg/jpeg, png, tiff, bmp). Accepts an `accounting_standard` (IFRS / NCT / SYSCOHADA; default NCT).
- **Idempotency-Key** header dedupes partner retries — a repeat returns the original job.
- **GET /documents** — tenant-scoped, paginated list, filterable by state.
- **GET /documents/{id}** — detail with latest job summary (status, tier, QCS).
- **GET /documents/{id}/result** — full serialized `ExtractionResult` (text, tables, canonical tables, QCS, tier, confidence, metadata).
- **Worker task** — retry with exponential backoff (max 3), `acks_late`, soft-time-limit handling, clean skip for cancelled / deleted jobs; failure persisted only after retries exhaust. Pipeline lazy-loaded once per prefork child.

2. Identity, Tenancy & Authorization

- **Dual auth on one Principal** — human bearer token or partner X-API-Key resolve through the same dependency.
- **Human auth** via fastapi-users with a revocable Redis token strategy — logout deletes the server-side token (instant revocation), 30-min lifetime. Argon2 password hashing; policy (≥ 10 chars, no email-name).
- **Partner API keys** — one-time plaintext at mint; only an Argon2 hash stored, with a short indexed prefix for fast lookup before constant-time verify. Scopes + expiry + active flag.
- **Scope-based RBAC** via `require_scopes(...)`. Roles → scopes (admin, analyst, viewer); scopes: `documents:submit/read`, `hitl:write`, `feedback:write`, `analytics:read`, `admin`.
- **Multi-tenancy** — every query scoped by `org_id`; cross-org access returns 404.

3. HITL Review Workflow

- **GET /review/queue** — draft documents needing review, annotated with `hitl_required` + QCS.
- **GET /documents/{id}/validation** — validation view with V1–V4 report snapshot, derived conflicts and flagged (non-green) cells from the canonical tables.
- **POST /assign** — assign a reviewer.
- **PATCH /cells** — record human corrections as append-only rows; If-Match concurrency guard.
- **State machine (POST /transition)** — Draft → Validated / Rejected, Validated → Archived; illegal transitions rejected (409); optimistic locking (`version_id`) → 409 on concurrent edits.
- **Full audit trail** — every mutation writes an immutable `AuditEvent`; `GET /audit` returns it.

4. Feedback Loop (compounding-accuracy engine)

- A HITL correction with a canonical mapping auto-seeds an approved `VariationCandidate` — closing the loop into the DB.

- **/feedback/variations** — list + approve / reject. Approval promotes and bridges into the pipeline's hot-reload VariationLogger so future extractions benefit.
- **/feedback/rules/flags** — validation-rule pass rates and critical-failure rate across jobs.
- **/feedback/quality** — KPIs: automation rate, HITL rate, avg QCS, correction rate, totals.
- **/feedback/ground-truth** — exports corrections as a JSONL labeled dataset (golden-set / calibration source).

5. Configuration & Reference Data

- **Per-org config** GET / PUT (thresholds, default standard, limits) — tunable without redeploy (PUT requires admin).
- **/standards** — IFRS / NCT / SYSCOHADA catalog.
- **/nomenclature** — canonical accounting term dictionary, filterable by statement type.

6. Partner Administration

- Create partner orgs; mint API keys (one-time plaintext, scoped, optional expiry); revoke keys (soft-deactivate). All admin-scoped.

7. Webhooks

- Register callback endpoints with an SSRF guard (rejects non-http(s), localhost, private / loopback / link-local / reserved IPs, .internal / .local). HMAC signing secret returned once, hashed at rest.
- List endpoints, list deliveries (observability), and replay a delivery. Actual signed delivery with retry / backoff is the worker's job on document.validated.

8. Cross-cutting Platform

- RFC 9457 application/problem+json error envelopes everywhere, with request_id correlation.
- Request-ID middleware — accepts or mints X-Request-ID, echoed on response.
- Per-principal rate limiting (fastapi-limiter / Redis), bucketed by key / token / IP; degrades to no-op when Redis is absent; submit path tighter (30 / 60s).
- Prometheus /metrics (fresh registry per app); /healthz + /readyz probes.
- Object storage abstraction — MinIO / S3 when configured, local filesystem fallback for dev/tests.
- Typed settings via pydantic-settings (FINALZE_* env, .env, Docker/K8s /run/secrets), validated Postgres / Redis / RabbitMQ DSNs. CORS, docs gating, proxy root_path support.

9. Data Model & Lineage

Org, User, ApiKey, OrgConfig, Document, Job, Validation, Correction, VariationCandidate, AuditEvent, WebhookEndpoint / Delivery. UUID PKs; JSON columns portable across SQLite (tests) and Postgres / JSONB (prod). Jobs carry lineage fields (pipeline_version, engine_versions, config_used) for reproducibility; Documents carry forward-seam analytical keys (entity, period) for future KG / warehouse ingestion.

10. Deployment & Clients

- Docker Compose stack: postgres, redis, rabbitmq, minio, api, worker, flower, traefik, optional GPU worker. Split requirements-api.txt / requirements-worker.txt.
- Bootstrap (python -m api.bootstrap) — idempotent create_all + initial superuser for first-run compose.
- Python SDK (client/sdk.py) — submit_and_wait(), both auth modes, injectable transport for tests; wired into the Streamlit UI and the pipeline CLI (--remote).

Maturity Caveats

- ! **No Alembic migration exists yet** — schema relies on create_all / bootstrap; alembic/versions/ only has a README.
- ! **Webhook signed delivery + retry** is referenced as the worker's responsibility but is not yet implemented (only config / observability endpoints + the extraction task exist).
- ! **Upload hardening** is allowlist + size-cap only; AV / content-sniffing is a documented follow-up.

Generated from source review of src/api, src/worker, and src/client.